

Laboratory Quiz / Viva Answers

Short answers to the quiz questions of all 12 experiments — for hand-written submission

Particulars	Details
Name of the student	
Enrolment No. · Branch	Robotics and Automation
Subject	Microcontroller and Microprocessor (BE05041011)
Experiments covered	All 12 (8085 · 8051 · AVR)
Date	

Experiment 1 — Introduction to Microprocessor (8085)

1. What are the various registers available in 8085?

Ans. The 8085 has several registers. The **Accumulator (A)** is an 8-bit register that holds one operand and the result of every arithmetic and logical operation. There are six 8-bit **general-purpose registers — B, C, D, E, H and L** — which can be used individually or as the 16-bit pairs BC, DE and HL to hold addresses or data. The **Flag register** holds the status flags. Two 16-bit registers are provided: the **Program Counter (PC)**, which points to the next instruction, and the **Stack Pointer (SP)**, which points to the top of the stack. There are also internal temporary registers (W, Z, temp) and the Instruction Register, which are not accessible to the programmer.

2. What are the various flags used in 8085?

Ans. The flag register of the 8085 has **five flags** that indicate the status of the result of an operation. **Sign (S)** is set when the result is negative (MSB = 1). **Zero (Z)** is set when the result is zero. **Auxiliary Carry (AC)** is set when there is a carry from bit 3 to bit 4, and is used in BCD/DAA operations. **Parity (P)** is set when the result has an even number of 1s. **Carry (CY)** is set when an operation produces a carry or borrow out of the MSB. These flags are tested by the conditional branch instructions.

3. What is stack pointer?

Ans. The **Stack Pointer (SP)** is a 16-bit register that holds the address of the **top of the stack**, which is a reserved area of read/write memory used on a last-in-first-out (LIFO) basis. It is used to temporarily store the contents of registers and return addresses during PUSH, POP, CALL and RET operations and during interrupts. When data is pushed, SP is decremented; when data is popped, SP is incremented.

4. What is program counter?

Ans. The **Program Counter (PC)** is a 16-bit register that holds the **memory address of the next instruction** to be fetched and executed. After each instruction byte is fetched, the PC is automatically incremented so that the program executes in sequence. During a jump, call or interrupt, a new address is loaded into the PC so that control is transferred to a different part of the program.

5. Define interrupt.

Ans. An **interrupt** is a signal that temporarily halts the normal execution of the main program and transfers control to a special routine called the **Interrupt Service Routine (ISR)**. The processor saves the current program status (return address) on the stack, executes the ISR, and then returns to continue the main program from where it left off. Interrupts allow the processor to respond quickly to important external or internal events without continuously checking (polling) for them.

6. What are the hardware & software interrupts?

Ans. **Hardware interrupts** are generated by external devices through the physical interrupt pins of the 8085 — **TRAP, RST 7.5, RST 6.5, RST 5.5 and INTR**. TRAP is non-maskable and has the highest priority; the others are maskable. **Software interrupts** are generated internally by executing an instruction in the program — **RST 0 to RST 7** — each of which transfers control to a fixed vector address. Thus hardware interrupts come from outside the chip, while software interrupts are written into the program by the programmer.

7. What is the clock frequency for 8085?

Ans. The 8085 normally operates at an internal clock frequency of about **3 MHz** (the maximum rating is around 3.072 MHz). The device has an **on-chip clock generator**; a crystal of about **6 MHz** is connected across the X1 and X2 pins and is internally **divided by two** to give the operating frequency. The clock determines the speed at which instructions are executed.

8. What is ALE? Explain the function of ALE in 8085.

Ans. **ALE stands for Address Latch Enable**. In the 8085 the lower-order address lines and the data lines share the same pins (**AD0-AD7**), i.e. they are multiplexed to save pins. ALE is used to **demultiplex** this bus. During the first clock state (T1) of each machine cycle, ALE goes HIGH and the lines carry the lower byte of the address; this address is latched into an external latch (such as a 74LS373) at the falling edge of ALE. For the rest of the cycle ALE is LOW and the same lines are then used to carry data. In short, ALE tells the external latch when a valid address is present on the multiplexed bus.

Experiment 2 — Data Transfer Instructions

1. Write the names of addressing modes.

Ans. An addressing mode is the method by which the operand of an instruction is specified. The 8085 supports **five addressing modes**: (i) **Immediate** — the operand (data) is given directly in the instruction, e.g. **MVI A,05H**; (ii) **Direct** — the 16-bit memory address of the operand is given, e.g. **LDA 2000H**; (iii) **Register** — the operand is in a register, e.g. **MOV A,B**; (iv) **Register Indirect** — the address of the operand is held in a register pair, e.g. **MOV A,M** (address in HL); and (v) **Implied (Implicit)** — the operand is understood and not written explicitly, e.g. **CMA, RAL**.

2. List any three data transfer instructions with immediate addressing mode.

Ans. In immediate addressing the data is part of the instruction itself. Three examples are: **MVI A, 05H** (loads 05H into the accumulator), **MVI B, 0FH** (loads 0FH into register B) and **LXI H, 2000H** (loads the 16-bit value 2000H into the HL register pair). In each case the value to be loaded is written directly in the instruction.

3. Explain the operation of PUSH & POP instruction with a suitable program.

Ans. The **PUSH** instruction stores the contents of a register pair onto the stack. The stack pointer is first decremented and the higher byte is stored, then it is decremented again and the lower byte is stored, so SP finally reduces by 2. The **POP** instruction does the reverse — it retrieves two bytes from the stack back into a register pair and increases

SP by 2. They work on a LIFO basis and are used to save and restore data. Example: LXI SP, 27FFH (initialise stack); LXI B, 1234H; PUSH B (saves BC on the stack); POP D (loads the saved value into DE, so DE = 1234H).

Experiment 3 — Arithmetic and Logical Instructions

1. What is the purpose of the ADD instruction in 8085?

Ans. The **ADD** instruction performs 8-bit binary addition. It adds the contents of a specified register or memory location to the **accumulator** and stores the result back in the accumulator ($A = A + R$). It affects all the status flags according to the result. For example, ADD B adds the contents of register B to A; ADD M adds the contents of the memory location pointed to by HL.

2. Which 8085 instruction is used for logical AND operations?

Ans. The logical AND is performed by the **ANA** and **ANI** instructions. **ANA R / ANA M** performs a bit-by-bit AND of the accumulator with a register or memory, while **ANI data** performs AND with an immediate 8-bit value. The result is stored in the accumulator. The AND operation is commonly used to **mask (clear) selected bits** of a byte.

3. Which instruction decrements a register or memory location?

Ans. The **DCR** instruction decrements the contents of an 8-bit register or memory location by one, e.g. **DCR B** or **DCR M**. It affects all flags except the carry flag, and is frequently used as a loop counter. For decrementing a 16-bit register pair the **DCX** instruction is used (e.g. DCX H), which does not affect any flags.

4. What is the role of the CMP instruction?

Ans. The **CMP** (compare) instruction compares the contents of a register or memory location with the accumulator by internally **subtracting** it from the accumulator. The result is **not stored** anywhere — only the flags are affected. If $A = \text{operand}$ the Zero flag is set; if $A < \text{operand}$ the Carry flag is set; if $A > \text{operand}$ both flags are reset. It is used in decision-making before conditional branches.

5. Which instruction is used to clear a specific bit in a register/memory?

Ans. A specific bit can be cleared by performing a logical AND with a suitable **mask** using the **ANI** (or ANA) instruction. The mask must contain a **0 at the bit position to be cleared** and 1 at all other positions, so that only that bit becomes 0 while the rest remain unchanged. For example, **ANI 0FEH** clears bit 0 of the accumulator (mask = 1111 1110).

6. What is the purpose of the XRA instruction?

Ans. The **XRA** instruction performs a bit-by-bit **Exclusive-OR (XOR)** between the accumulator and a register or memory, storing the result in the accumulator. XOR gives 1 only when the two bits differ. A very common use is **XRA A**, which XORs the accumulator with itself and thereby **clears the accumulator to 00H** and resets the carry flag — a fast way to initialise A to zero.

7. Which instruction performs addition with carry?

Ans. Addition with carry is performed by the **ADC** and **ACI** instructions. **ADC R / ADC M** adds the register/memory contents **plus the carry flag** to the accumulator, and **ACI data** adds an immediate value plus the carry. These instructions are essential for **multi-byte (16-bit or larger) addition**, where the carry generated from the lower byte must be added to the next higher byte.

1. What is the difference between conditional & unconditional branching instructions?

Ans. An **unconditional branch** always transfers control to the specified address, regardless of any condition — examples are **JMP**, **CALL** and **RET**. A **conditional branch** transfers control only if a particular **flag condition is satisfied**; otherwise execution continues with the next instruction in sequence. Examples are **JZ / JNZ** (jump if zero / not zero), **JC / JNC** (jump if carry / no carry) and **JP / JM** (jump if positive / minus). Conditional branches are what allow decision-making and loops to be implemented.

2. What is the difference in operation of JMP & CALL instruction?

Ans. The **JMP** instruction simply loads a new address into the program counter and transfers control there; it **does not save any return address**, so the program cannot automatically come back. The **CALL** instruction is used to invoke a subroutine — it first **pushes the return address (the address of the next instruction) onto the stack** and then jumps to the subroutine. When a **RET** instruction is executed at the end of the subroutine, the saved address is popped back into the PC and control returns to the main program. Thus CALL/RET support reusable subroutines, whereas JMP is a one-way transfer.

Experiment 5 — Data Transfer Programming in 8051

1. List the available software for the microcontroller practical.

Ans. Several software tools are available for writing, assembling, simulating and testing 8051/AVR programs. Common ones are **Keil µVision** (IDE and C/assembler for 8051), **SimulIDE** (circuit and microcontroller simulator), **Proteus** (schematic capture and simulation), **MIDE-51** and **EdSim51** (8051 simulators), and the **AVR Studio / Microchip Studio** with the AVR-GCC toolchain for AVR controllers. These tools let the student write code, generate the HEX file and verify operation without actual hardware.

2. Describe the dissimilarities between internal and external memory in the 8051.

Ans. **Internal (on-chip) memory** of the 8051 consists of **128/256 bytes of RAM and 4 KB of ROM** built inside the chip. It is accessed directly and quickly using instructions like MOV, but its size is limited. **External memory** is memory added **outside the chip** and can be extended up to **64 KB of RAM and 64 KB of ROM**. It is accessed more slowly through Port 0 and Port 2 (which carry the address/data bus) using the **MOVX** instruction for external RAM and **MOVC** for external code memory. In short, internal memory is small and fast, while external memory is large but slower and needs extra hardware.

Experiment 6 — Arithmetic Instruction Programming in 8051

1. Describe the purpose of studying data transfer and arithmetic operations on external memory.

Ans. The purpose is to learn how a microcontroller **stores data in and retrieves data from external memory** and then performs arithmetic (addition, subtraction, etc.) on that data. In real applications the on-chip RAM is often too small to hold all the data, so external RAM is used. Studying these operations teaches how to use the **MOVX** instruction to move data to and from external memory, how to address it correctly, and how to process large amounts of data — skills needed in data-logging and other memory-intensive embedded systems.

2. Define external memory.

Ans. **External memory** is memory that is connected **outside the microcontroller chip** to extend its storage capacity. The 8051 can address up to **64 KB of external RAM** and **64 KB of external program (code) memory**. It is interfaced through Port 0 (lower address/data, multiplexed) and Port 2 (higher address) and is accessed with the **MOVX** instruction (for data memory) and the **MOVC** instruction (for code memory). It is used when the limited internal memory is not sufficient for the program or data.

Experiment 7 — Boolean & Logical Instructions Programming (8051)

1. Explain the purpose of the ANL instruction in 8051 programming.

Ans. The **ANL** instruction performs a bit-by-bit **logical AND** between the destination operand (the accumulator or a directly addressed byte) and the source operand, and stores the result in the destination. Since ANDing a bit with 0 forces it to 0 and ANDing with 1 leaves it unchanged, ANL is mainly used to **mask or clear selected bits** of a byte or port while keeping the other bits untouched — for example, to turn off particular output pins of a port.

2. Illustrate how to clear a bit in a byte using the 8051.

Ans. A particular bit in a byte is cleared by **ANDing the byte with a mask** that has a 0 at the bit to be cleared and 1 at every other position. For example, **ANL A, #0FEH** clears bit 0 of the accumulator because the mask 1111 1110 leaves all other bits unchanged. For a **bit-addressable** location, the simpler **CLR** instruction can be used directly — e.g. **CLR P1.0** clears pin P1.0, and **CLR ACC.3** clears bit 3 of the accumulator.

Experiment 8 — Blinking LED using 8051

1. What is the 8051 microcontroller?

Ans. The 8051 is a popular **8-bit single-chip microcontroller** originally developed by Intel. On a single chip it integrates a CPU, **4 KB of ROM**, **128 bytes of RAM**, **four 8-bit input/output ports (P0-P3)**, **two 16-bit timers/counters**, a full-duplex **serial communication port (UART)** and an **interrupt system** with five sources. Because it combines the processor and all these peripherals in one device, it is widely used in embedded control applications.

2. What is the function of Port 1?

Ans. **Port 1 (P1)** is an **8-bit bidirectional general-purpose I/O port** of the 8051 with internal pull-up resistors. Each of its pins (P1.0–P1.7) can be independently used as an input or an output. Unlike Port 0 and Port 2, Port 1 is dedicated to I/O and is not used for the external address/data bus, which makes it convenient for connecting devices such as LEDs and switches.

3. Which pin of Port 1 is used for the LED?

Ans. In this experiment the LED is connected to **pin P1.0**, the least-significant bit of Port 1. The program drives this pin HIGH and LOW with a delay in between to make the LED blink. (Any other pin of Port 1 could be used in the same way.)

4. Why is a resistor connected in series with the LED?

Ans. A resistor is connected in series with the LED to act as a **current-limiting resistor**. An LED has very low resistance, so without it the current would be too high and would burn out the LED or overload the port pin. The resistor (typically **220 Ω to 330 Ω**) limits the current to a safe value (around 10–20 mA) while still letting the LED glow brightly.

5. What is the purpose of the crystal oscillator?

Ans. The **crystal oscillator** connected across the XTAL1 and XTAL2 pins provides a stable and accurate **clock signal** to the microcontroller. This clock determines the **operating speed and the timing** of all internal operations, including instruction execution and the delays generated in the program. Without the clock the microcontroller cannot run.

6. Why is 11.0592 MHz commonly used with the 8051?

Ans. A crystal of **11.0592 MHz** is commonly used because, after the internal division, it divides **evenly to give the exact standard baud rates** (such as 9600, 4800, 2400 bps) required for serial (UART) communication. A "round" frequency like 12 MHz gives small errors in the baud rate, whereas 11.0592 MHz gives error-free baud rates, which is why it is preferred whenever serial communication is used.

7. What is the purpose of ORG 0000H?

Ans. **ORG** is an assembler directive meaning "origin". **ORG 0000H** tells the assembler to place the program code starting from **memory address 0000H**. This is important because when the 8051 is reset, the program counter is set to 0000H, so execution always begins from that location. Hence the first instruction of the program is placed there.

8. How to create a hex file?

Ans. The HEX file is created using an IDE or assembler such as **Keil µVision**. The steps are: write the source program (in assembly or C), create a project and add the source file, select the correct target device, and then **build/compile the project**. The assembler or compiler translates the source into machine code and automatically generates the **.hex file**, which is then loaded into the microcontroller or simulator.

9. How many ports does the 8051 have?

Ans. The 8051 has **four 8-bit input/output ports — P0, P1, P2 and P3** — giving a total of **32 I/O lines**. Each port can be used for general-purpose I/O, and some ports have additional (alternate) functions: P0 and P2 are used for the external address/data bus, while P3 pins have special functions such as serial I/O (RxD, TxD), interrupts (INT0, INT1), timer inputs and the read/write control lines.

Experiment 9 — AVR Arithmetic & Logical Operations (Viva)

1. What is the purpose of the LDI instruction?

Ans. The **LDI (Load Immediate)** instruction loads an **8-bit constant value directly into a register**. It can only be used with the upper registers **R16 to R31**. For example, **LDI R16, 0x05** loads the value 05H into register R16. It is one of the most frequently used AVR instructions for initialising registers with known values before processing.

2. Where is the result of the MUL instruction stored?

Ans. The **MUL** instruction multiplies two 8-bit unsigned numbers and produces a 16-bit result. This result is always stored in the **register pair R1:R0** — the **high byte in R1** and the **low byte in R0**. Because R0 and R1 are used automatically, the programmer must read the product from them (and often save them) immediately after the multiplication.

3. Which AVR instruction performs XOR?

Ans. The **EOR (Exclusive OR)** instruction performs a bit-by-bit logical XOR between two registers and stores the result in the destination register, e.g. **EOR R16, R17**. A useful special case is **EOR R16, R16**, which XORs a register with itself and thereby **clears it to zero**.

4. What does the COM instruction do?

Ans. The **COM** instruction takes the **one's complement** of a register — it **inverts every bit**, changing each 0 to 1 and each 1 to 0 (equivalent to $R = 0xFF - R$). For example, if a register holds 0x0F, after COM it holds 0xF0. It is used in logical/bit-manipulation operations and to form complements.

5. What is a HEX file?

Ans. A **HEX file** is a text file in the **Intel HEX format** that contains the program's **machine code represented in hexadecimal**, along with address and checksum information. It is the output of the assembler/compiler and is the file that is actually **loaded (burned) into the microcontroller's program (flash) memory** or into a simulator to run the program.

Experiment 10 — Read Push-Button Switch & Display on LED (8051)

1. Which port is used to read the push button?

Ans. The push button is read through **Port 1** — specifically pin **P1.0**. One terminal of the switch is connected to P1.0 and the other to ground. Because Port 1 has internal pull-ups, P1.0 stays HIGH when the switch is open and goes LOW when it is pressed, so the program can sense the switch by reading this pin.

2. Which pin is used to control the LED?

Ans. The LED is controlled through pin **P2.0**. The program sets P2.0 HIGH to turn the LED ON and clears it LOW to turn the LED OFF, according to the status read from the push button.

3. Why is FFH written to Port 1?

Ans. The instruction **MOV P1, #0FFH** writes all 1s to Port 1 so that it is configured to work as an **input port**. In the 8051, to use a port pin as an input it must first be written with a 1; this turns on the internal pull-up and lets the external switch pull the pin LOW. Hence writing FFH makes P1.0 able to correctly read the push-button status.

4. What happens to P1.0 when the switch is pressed?

Ans. When the push button is pressed, P1.0 is **connected to ground**, so it reads **logic 0 (LOW)**. The program detects this LOW level and in response **turns the LED at P2.0 ON**. When the button is released, the internal pull-up makes P1.0 read logic 1 again and the LED is turned OFF.

5. What is the function of the JB instruction?

Ans. The **JB (Jump if Bit set)** instruction tests a specified bit and **jumps to the given label if that bit is 1**; if the bit is 0, execution continues with the next instruction. In this program **JB P1.0, NOT_PRESSED** checks the switch: if P1.0 = 1 (not pressed) it jumps to the NOT_PRESSED routine and keeps the LED off; otherwise it falls through to turn the LED on. It is a bit-testing conditional branch.

Experiment 11 — Interfacing Buzzer with AVR Board

1. What is the function of a buzzer in an embedded system?

Ans. A **buzzer** is an **electro-acoustic output device** that converts an electrical signal into **sound**. In embedded systems it is used to give **audible indications** — for example alarms, warnings, key-press beeps, or status/confirmation tones — so that the user is alerted without having to look at a display. It is controlled by a digital output pin of the microcontroller.

2. Why is PB0 configured as an output in this experiment?

Ans. The buzzer is connected to pin **PB0** of Port B, and the microcontroller has to **send signals to drive it**. A pin can drive an external device only when it is set as an **output**. Therefore PB0 is configured as an output (using the DDRB register) so that the program can make it HIGH or LOW to switch the buzzer ON and OFF.

3. What is the purpose of the DDRB register?

Ans. **DDRB (Data Direction Register for Port B)** decides the **direction of each pin of Port B** — whether it is an input or an output. Writing **1** to a bit makes that pin an **output**, and writing **0** makes it an **input**. In this experiment the statement **DDRB |= (1 << PB0)** sets the PB0 bit to 1, configuring PB0 as an output so it can drive the buzzer.

4. What is the difference between an active buzzer and a passive buzzer?

Ans. An **active buzzer** has a **built-in oscillator circuit**, so it produces sound automatically when a steady **DC voltage** is applied — the microcontroller only needs to turn it ON or OFF. A **passive buzzer** has **no internal oscillator**; it behaves like a small speaker and needs an **external AC or PWM signal** of a suitable frequency to produce sound, which allows different tones but requires more programming. This experiment uses an active buzzer, so a simple HIGH/LOW on PB0 is enough.

5. What is the function of PORTB in this program?

Ans. The **PORTB** register sets the **output logic level** of the Port B pins (once they are configured as outputs by DDRB). In this program, **PORTB |= (1 << PB0)** makes PB0 HIGH and switches the **buzzer ON**, while **PORTB &= ~(1 << PB0)** makes PB0 LOW and switches the **buzzer OFF**. By alternating these two with a delay, the buzzer beeps ON and OFF repeatedly.

Experiment 12 — Interfacing 7-Segment Display with AVR Board

1. What is a 7-segment display?

Ans. A **7-segment display** is a simple numeric display made up of **seven LED segments arranged in the shape of the digit 8 (named a, b, c, d, e, f and g)**, together with an optional **decimal point (dp)**. By lighting up particular combinations of these segments, the display can show the **decimal digits 0-9** and a few letters. It is widely used in digital clocks, meters and counters because it is cheap and easy to drive.

2. What is the difference between common-cathode and common-anode displays?

Ans. In a **common-cathode** display, all the **cathodes of the segment LEDs are joined together and connected to ground**; a segment lights up when its anode is driven **HIGH (logic 1)**. In a **common-anode** display, all the **anodes are joined together and connected to +V**; a segment lights up when its cathode is driven **LOW (logic 0)**. In short, the two types are driven with opposite logic levels. This experiment uses a common-cathode display, so the segment codes use 1s for the ON segments.

3. Why are 330 Ω resistors used with the LED segments?

Ans. Each segment of the display is an LED, and LEDs must not be driven directly because they would draw excessive current. The **330 Ω resistors are connected in series with each segment as current-limiting resistors**, which restrict the current to a safe value (about 10–15 mA per segment). This protects both the LED segments and the microcontroller port pins while keeping the display bright.

4. Which ATmega32 port is used in this experiment?

Ans. **PORTD** of the ATmega32 is used to drive the display. The eight pins **PD0 to PD7** are connected to the segments **a, b, c, d, e, f, g and dp** respectively. The program writes the appropriate segment code to PORTD to show each digit.

5. What does DDRD = 0xFF do?

Ans. The statement **DDRD = 0xFF** writes 1 to all eight bits of the Data Direction Register for Port D, which **configures every pin of Port D (PD0-PD7) as an output**. This is required because all eight pins must drive the segments of the display.

6. Why is F_CPU defined as 8 MHz?

Ans. **F_CPU** tells the compiler the **clock frequency of the microcontroller**, which here is **8 MHz (8000000 Hz)**. The delay functions such as **_delay_ms()** use this value to calculate the correct number of CPU cycles for a given delay. If F_CPU is set wrongly, the delays (and therefore the time each digit is shown) will be inaccurate.

7. What hexadecimal code is used to display digit 8?

Ans. The digit **8** requires **all seven segments (a-g) to be ON**. For the common-cathode display used here, this corresponds to the hexadecimal code **0x7F** (binary 0111 1111), which is written to PORTD to light every segment and show the figure 8.